

System Architecture of an NLP Search Engine

Tokenization · Inverted index · TF-IDF · Okapi BM25

Hamza Abdul Karim · F23607046 — May 2026

ABSTRACT

We describe the architecture of a small information retrieval engine implemented in pure Python. The system follows the classical sparse retrieval pipeline: documents are tokenized, normalized, and indexed into an inverted index; queries are parsed into structured form and scored against candidate documents with TF-IDF, Okapi BM25, or semantic search (TF-IDF vectors); finally, matching documents are filtered, snipped, and returned to the caller through a single facade. Three frontends share this facade: a command-line interface, a Flask web application with theme toggle/bookmarks/history, and a comprehensive JSON API with batch operations and Prometheus metrics. The discussion below introduces every component, its responsibilities, and its compile-time dependencies, accompanied by five reference diagrams.

1. Introduction

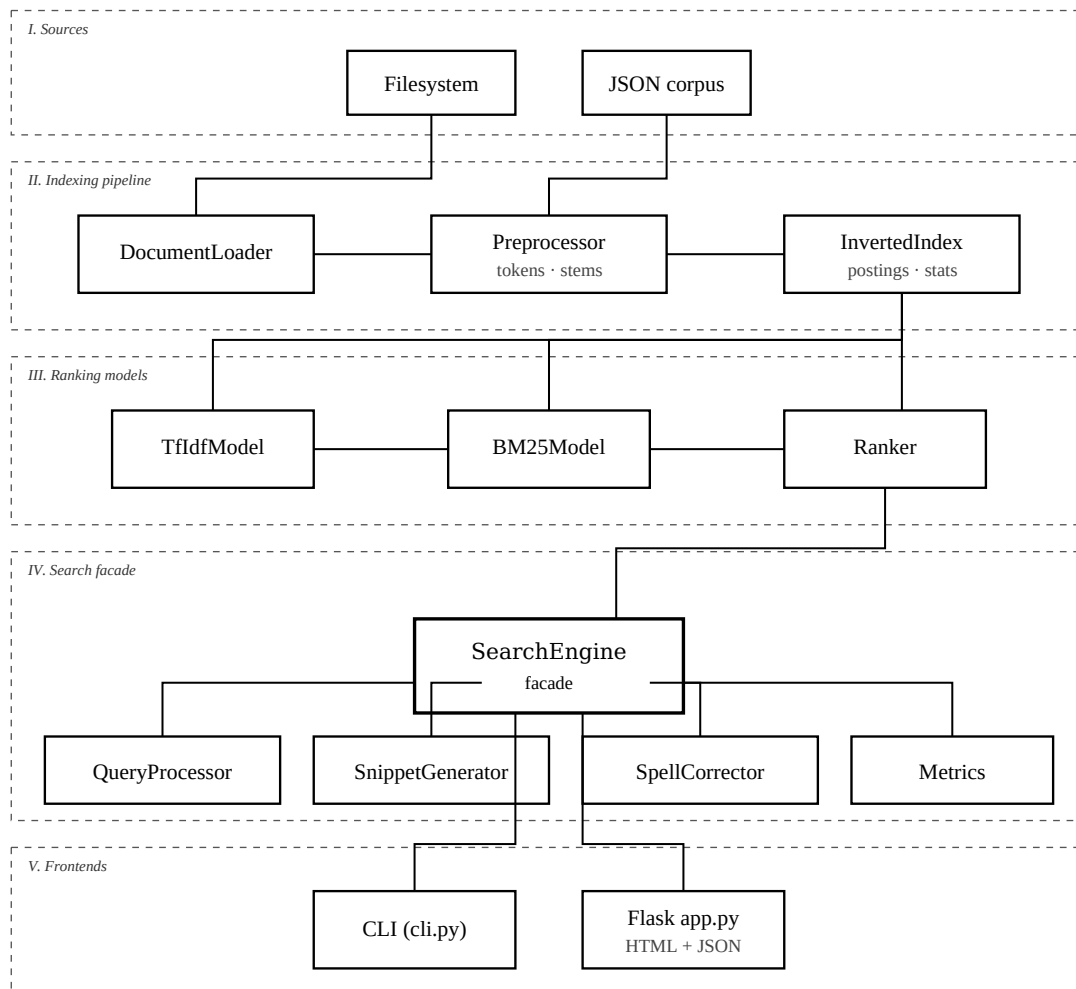
Modern web search systems combine sparse and dense retrievers, but the sparse layer rooted in lexical matching, document statistics, and carefully designed scoring functions remains the right starting point for understanding how search works. This document describes a minimal yet complete implementation of that layer, written from scratch in roughly two thousand lines of Python distributed across small, single-responsibility files. Each module is intentionally kept under two hundred lines so a reader can finish it in one sitting.

We begin with a high-level architectural map (Section 2), then follow the data through indexing (Section 3) and querying (Section 4). Section 5 documents the static class structure, Section 6 covers the web request flow, and Section 7 closes with a note on persistence.

2. Overall System Architecture

Conceptually, the engine is organised into five horizontal layers. Inputs arrive from a directory of text files or from a JSON corpus. An indexing pipeline transforms raw text into an inverted index. Two ranking models sit on top of that index, exposed through a thin facade. The search facade adds query parsing, snippet generation, and spell correction. Finally, two frontends — a CLI and a Flask app — consume the facade. Figure 1 shows the layout.

Figure 1. System architecture
Components and their compile-time dependencies



Solid arrows denote a "depends on" relation. Dashed boxes group components by responsibility.

Figure 1. The five layers of the search engine and their dependencies.

2.1 Module map

Each box in Figure 1 corresponds to one Python file under `src/`. The module-to-responsibility mapping is summarised in Table 1.

Module	Responsibility
<code>preprocessing.py</code>	Tokenizer, stop-word filter, Porter stemmer.
<code>document.py</code>	Document dataclass and serialization helpers.
<code>document_loader.py</code>	Read corpora from a directory or JSON file.
<code>indexer.py</code>	InvertedIndex with positional postings.
<code>tfidf.py</code>	TF-IDF model with cosine scoring.
<code>bm25.py</code>	Okapi BM25 with tunable k_1 / b .

Module	Responsibility
<code>semantic_search.py</code>	Semantic search with TF-IDF vectors.
<code>ranker.py</code>	Lazy facade over the ranking models.
<code>query_processor.py</code>	Parse +must, -must-not, phrases, booleans.
<code>snippet.py</code>	Best-window snippet with <mark> highlighting.
<code>spell_check.py</code>	Damerau-Levenshtein vocabulary lookup.
<code>analytics.py</code>	Search analytics tracking.
<code>autocomplete.py</code>	Query autocomplete suggestions.
<code>cache.py</code>	Search result caching layer.
<code>metrics.py</code>	IR metrics + SearchMetrics class.
<code>persistence.py</code>	SQLite backend (optional).
<code>search_engine.py</code>	End-to-end SearchEngine facade.

3. Indexing Pipeline

The indexing pipeline turns raw text into an inverted index in five stages, illustrated in Figure 2. Stage 1 is a filesystem path that points either at a directory of .txt files or at a single JSON file. The DocumentLoader (stage 2) reads the corpus and assigns sequential numeric IDs. The Preprocessor (stage 3) tokenizes each document with a regular expression, lowercases tokens, drops stop-words and tokens shorter than two characters, and finally applies a compact Porter stemmer. The Indexer (stage 4) inserts the resulting token sequence into the InvertedIndex, populating three persistent data structures along the way: the postings themselves, per-document lengths, and the document frequency for every term.

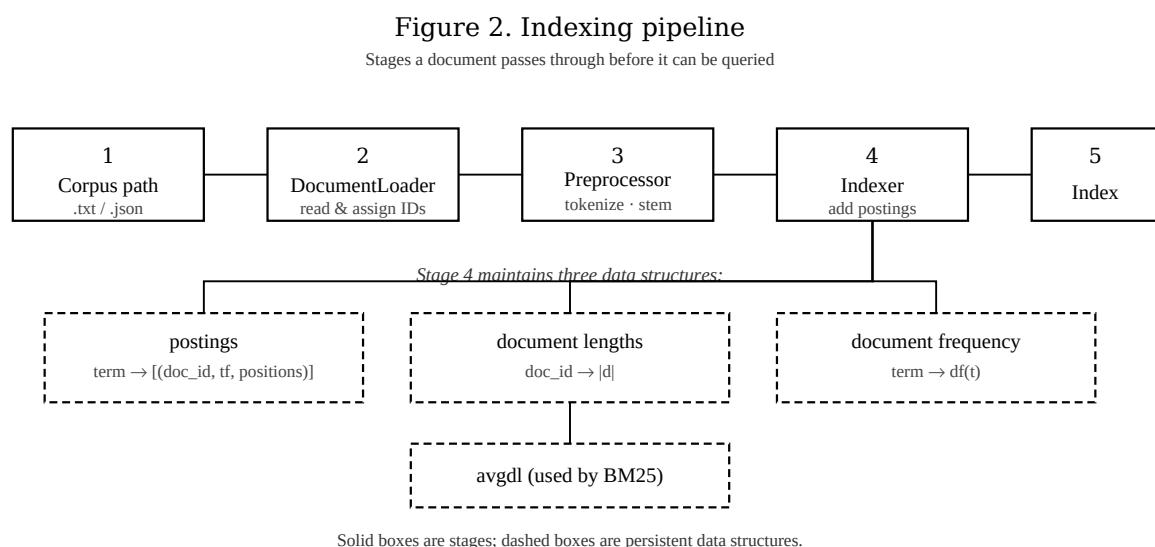


Figure 2. Stages and persistent data structures of the indexing pipeline.

From the per-document lengths the engine derives the average document length `avgdl`, which is consumed at scoring time by Okapi BM25's length normalization term (Section 5).

4. Query Lifecycle

Figure 3 traces a query from the user to a ranked list of results. The SearchEngine first hands the raw string to the QueryProcessor, which parses it into four buckets: required tokens (+term), optional tokens (bare terms), excluded tokens (-term), and quoted phrases. The Ranker is then invoked with the union of the positive bags. It walks the appropriate ranking model (BM25 by default) over the postings of every query term and produces a list of candidate (doc_id, score) pairs. The SearchEngine applies the phrase, must, and must-not filters; for each surviving document it asks the SnippetGenerator to pick the most relevant window in the original text and to wrap each match in <mark> tags. The result is a list of typed SearchResult records ready for display.

Figure 3. Query lifecycle (sequence diagram)

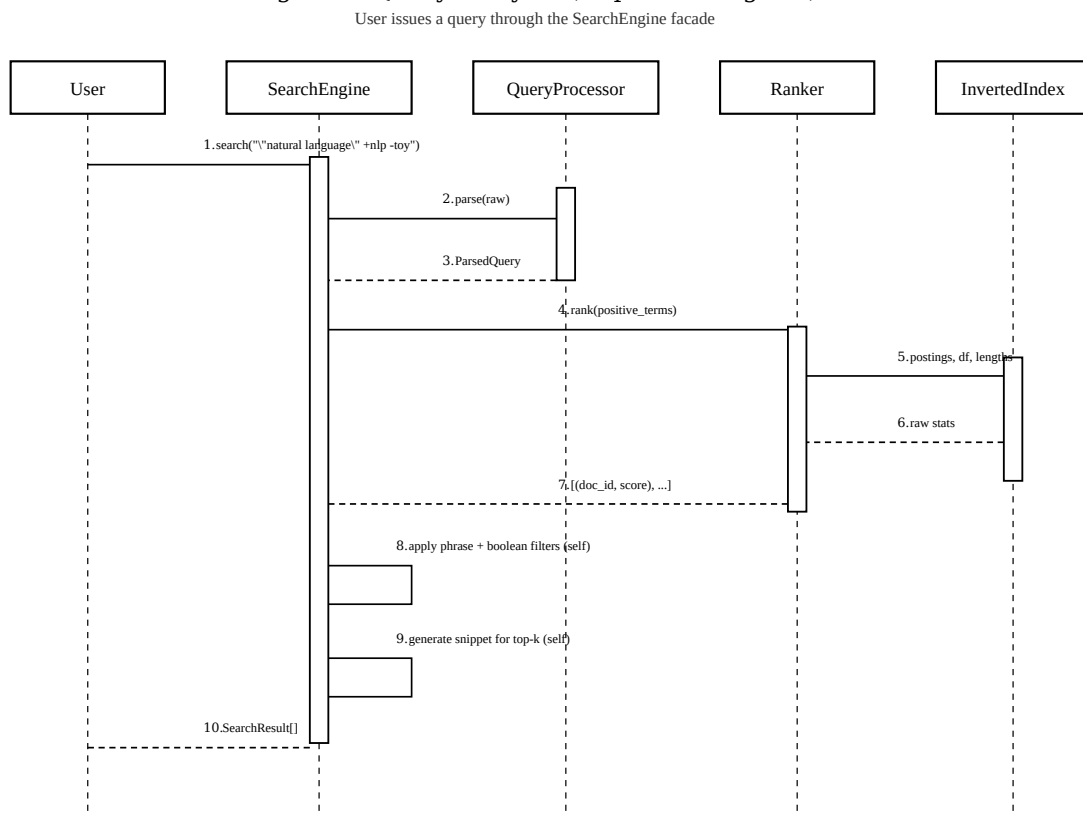


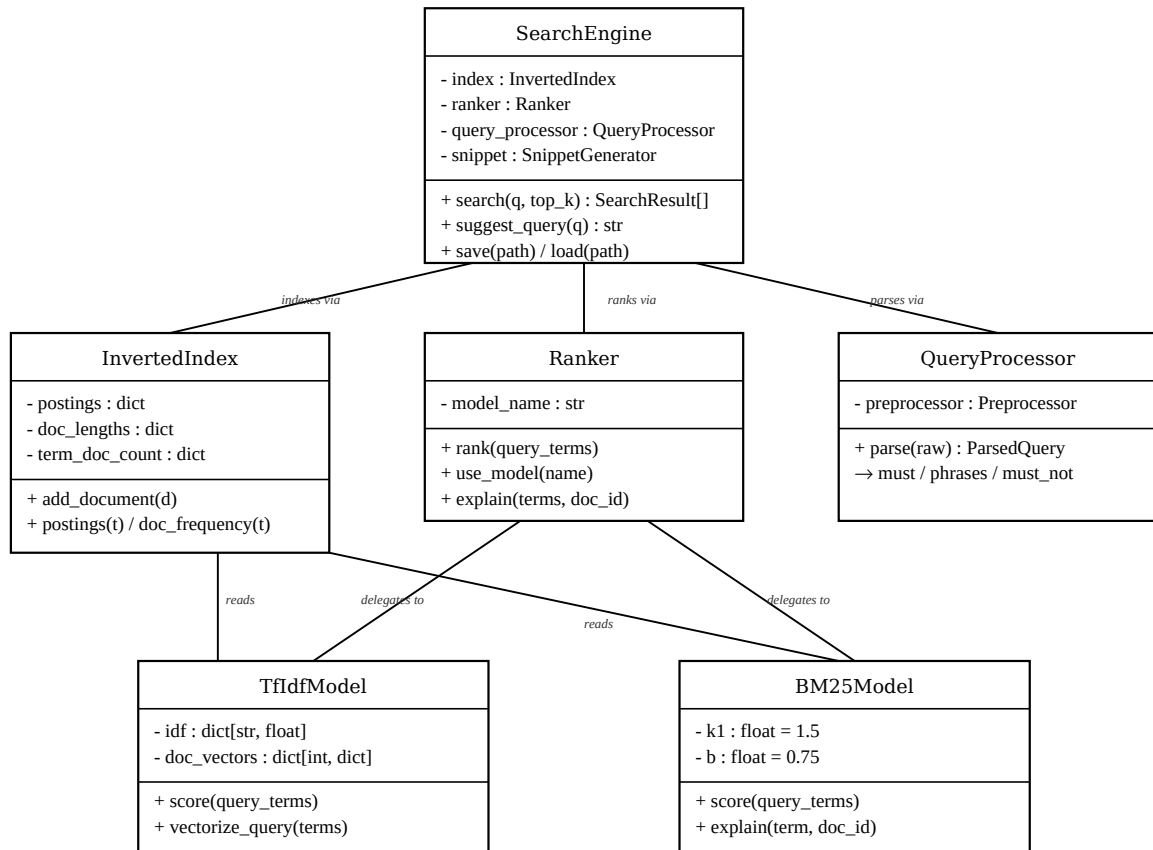
Figure 3. Sequence diagram of a single search request.

5. Class Structure

The static structure of the engine, simplified to UML, is shown in Figure 4. SearchEngine is the only class that external code needs to know about; the rest are reachable through it. Ranker delegates scoring to either TfIdfModel or BM25Model, both of which read from a shared InvertedIndex. QueryProcessor is the only component that depends on the Preprocessor directly — it shares the same tokenizer rules as the indexer to ensure that query and document terms live in the same vocabulary.

Figure 4. Class diagram (UML, simplified)

Core types of the engine and their associations



Hollow-headed arrows denote a directional association ("uses").

Compartment: name | attributes | operations.

Figure 4. Simplified UML class diagram of the engine's core types.

6. Web Request Flow

The Flask application creates exactly one `SearchEngine` instance at startup. Both the HTML routes (`/`, `/about`, `/search`) and the JSON endpoints (`/api/search`, `/api/stats`, `/healthz`) read from this single in-memory instance, as shown in Figure 5. No write endpoints exist, so the index is effectively immutable for the lifetime of the process; rebuilding the index requires a restart or a call to the build script.

Figure 5. Web request flow
HTML pages and JSON endpoints share the same engine instance

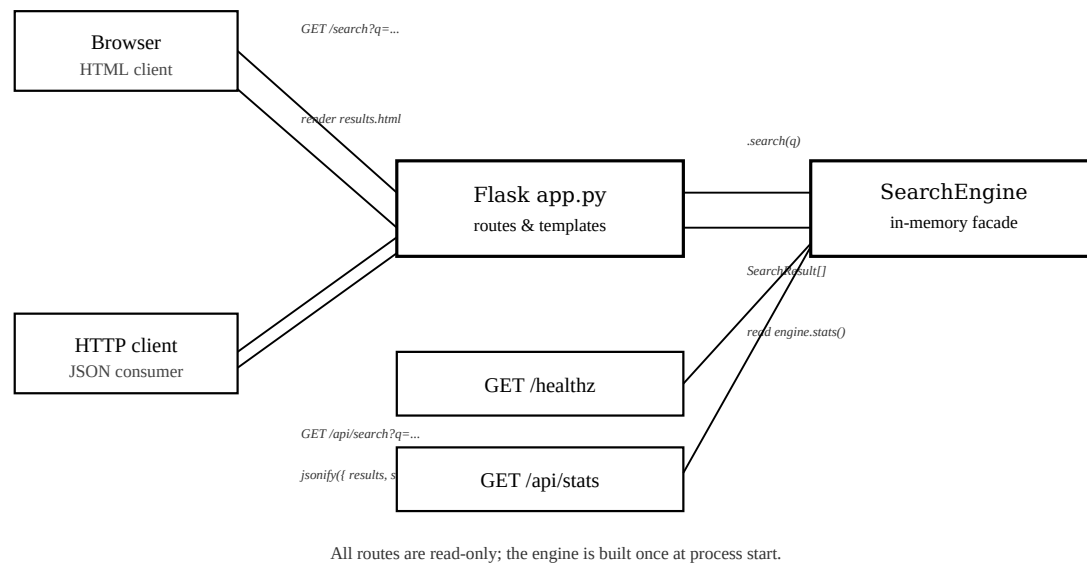


Figure 5. HTML and JSON request paths through the Flask app.

7. Persistence

`SearchEngine.save()` pickles the entire engine — index, ranker, preprocessor, and configuration — to `index_data/engine.pkl` by default. `SearchEngine.load()` reverses the operation. For larger corpora this approach should be replaced with a dedicated on-disk index format, for instance one file per posting list, without changing the public API.

8. References

- Manning, C. D., Raghavan, P., & Schütze, H. Introduction to Information Retrieval, Cambridge University Press, 2008.
- Robertson, S., & Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 2009.
- Porter, M. F. An Algorithm for Suffix Stripping. Program, 1980.